Experiment No. 6

Aim

To create, compile, deploy, and interact with a Ballot smart contract using Solidity and Remix IDE. To perform voting and delegation transactions, examine gas execution and transaction receipts, and observe real-time smart contract state transitions within the Remix VM environment.

Theory

A Solidity Ballot smart contract is a self-executing program deployed on an EVM-compatible blockchain. It supports **state-changing transactions**, which consume gas and modify blockchain storage, and **read-only calls**, which retrieve data without gas fees for the caller.

The Ballot contract implements decentralized voting with vote weighting and delegation. It uses a Vote struct to track voting weight, delegation, voting status, and selected proposal, and a Proposal struct to store proposal names and vote counts. A mapping manages voters by address, while a dynamic array stores proposals. The chairperson initializes the ballot and grants voting rights.

The contract uses require statements to enforce access control and valid voting conditions, including protection against delegation cycles. State variables are stored permanently in storage, while initialization parameters use temporary memory. Unlike standard implementations that use bytes32 for proposal names, this version uses dynamic stringtypes for human-readable proposal identification through functions such as winningProposal() and winnerName().

Steps

1. Open Remix IDE and navigate to the contracts directory within the default workspace.

2. Inspect the default Ballot.sol voting contract structure, analyzing voter mappings, proposal structs, and functions.

3. Embed student authorship and ownership proof constants (AUTHOR, ROLL_NO, BATCH) within the contract body.

4. Refactor the proposal representation from bytes32 to dynamic string across the Proposal struct, constructor parameters, and winnerName getter.

5. Open the Solidity Compiler tab, select compiler version pragma >=0.7.0 <0.9.0, and compile Ballot.sol.

6. Navigate to Deploy & Run Transactions, select the Remix VM environment, and input proposal names: ["Proposal A","Proposal B","Proposal C"].

7. Click Deploy to broadcast the deployment transaction, and inspect the resulting transaction receipt and contract address.

8. Interact with deployed contract getters to verify state initialization, checking chairperson, proposals, and the on-chain AUTHOR value.

9. Execute voting transactions and delegate operations between accounts, monitoring state updates in the Remix terminal.

10. Call winningProposal() and winnerName() to determine and verify the winning proposal from on-chain state.

Experiment No. 6

Code

```
// SPDX-License-Identifier: GPL-3.0
/*
 * ============================================================
 * AIM  : Creation and Deployment of Smart Contract and
 * Transactions using Solidity and Remix IDE
 * CO   : CO3, CO5, CO6
 * Author : Dipesh Nawani
 * Roll No: 19
 * Batch  : A
 * Modification (by Dipesh Nawani, Roll No 19, Batch A):
 * The original Remix Ballot.sol stores proposal names as
 * bytes32. In this version the proposal name has been
 * changed from bytes32 -> string, so that readable names
 * like "Proposal A" can be passed directly at deployment.
 * ============================================================
 */
pragma solidity >=0.7.0 <0.9.0;
/// @title Voting with delegation. (Modified by Dipesh Nawani | Roll No: 19 | Batch: A)
contract Ballot {
    // Author details stored on-chain as proof of ownership
    string public constant AUTHOR = "Dipesh Nawani";
    string public constant ROLL_NO = "19";
    string public constant BATCH = "A";
    // This declares a new complex type which will be used for variables later.
    // It will represent a single voter.
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted;  // if true, that person already voted
        address delegate; // person delegated to
        uint vote;   // index of the voted proposal
    }
    // This is a type for a single proposal.
```

Experiment No. 6

```
struct Proposal {
    string name;   // CHANGED: bytes32 -> string (Dipesh Nawani, Roll 19, Batch A)
    uint voteCount; // number of accumulated votes
}
address public chairperson;
// This declares a state variable that stores a 'Voter' struct for each possible address.
mapping(address => Voter) public voters;
// A dynamically-sized array of 'Proposal' structs.
Proposal[] public proposals;
/**
 * @dev Create a new ballot to choose one of 'proposalNames'.
 * @param proposalNames names of proposals (now as string[] instead of bytes32[])
 * Example input while deploying in Remix:
 * ["Proposal A","Proposal B","Proposal C"]
 */
constructor(string[] memory proposalNames) {
    chairperson = msg.sender;
    voters[chairperson].weight = 1;
    // For each of the provided proposal names,
    // create a new proposal object and add it to the end of the array.
    for (uint i = 0; i < proposalNames.length; i++) {
        // 'Proposal({...})' creates a temporary Proposal object and
        // 'proposals.push(...)' appends it to the end of 'proposals'.
        proposals.push(Proposal({
            name: proposalNames[i],
            voteCount: 0
        }));
    }
}
/**
 * @dev Give 'voter' the right to vote on this ballot. May only be called by 'chairperson'.
 * @param voter address of voter
 */
```

Experiment No. 6

```
function giveRightToVote(address voter) public {
    require(
        msg.sender == chairperson,
        "Only chairperson can give right to vote."
    );
    require(
        !voters[voter].voted,
        "The voter already voted."
    );
    require(voters[voter].weight == 0);
    voters[voter].weight = 1;
}
/**
 * @dev Delegate your vote to the voter 'to'.
 * @param to address to which vote is delegated
 */
function delegate(address to) public {
    Voter storage sender = voters[msg.sender];
    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");
    require(to != msg.sender, "Self-delegation is disallowed.");
    // Forward the delegation as long as 'to' also delegated.
    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        // We found a loop in the delegation, not allowed.
        require(to != msg.sender, "Found loop in delegation.");
    }
    Voter storage delegate_ = voters[to];
    // Voters cannot delegate to accounts that cannot vote.
    require(delegate_.weight >= 1);
    // Since 'sender' is a reference, this modifies 'voters[msg.sender]'.
    sender.voted = true;
    sender.delegate = to;
```

Experiment No. 6

```
        if (delegate_.voted) {
            // If the delegate already voted, directly add to the number of votes.
            proposals[delegate_.vote].voteCount += sender.weight;
        } else {
            // If the delegate did not vote yet, add to her weight.
            delegate_.weight += sender.weight;
        }
    }
    /**
     * @dev Give your vote (including votes delegated to you) to proposal 'proposals[proposal].name'.
     * @param proposal index of proposal in the proposals array
     */
    function vote(uint proposal) public {
        Voter storage sender = voters[msg.sender];
        require(sender.weight != 0, "Has no right to vote");
        require(!sender.voted, "Already voted.");
        sender.voted = true;
        sender.vote = proposal;
        // If 'proposal' is out of the range of the array,
        // this will throw automatically and revert all changes.
        proposals[proposal].voteCount += sender.weight;
    }
    /**
     * @dev Computes the winning proposal taking all previous votes into account.
     * @return winningProposal_ index of winning proposal in the proposals array
     */
    function winningProposal() public view
        returns (uint winningProposal_)
    {
        uint winningVoteCount = 0;
        for (uint p = 0; p < proposals.length; p++) {
            if (proposals[p].voteCount > winningVoteCount) {
                winningVoteCount = proposals[p].voteCount;
```

Experiment No. 6

```
            winningProposal_ = p;
        }
    }
}
/**
 * @dev Calls winningProposal() function to get the index of the winner contained in the proposals array
 * and then returns the name of the winner.
 * @return winnerName_ the name of the winner (CHANGED: bytes32 -> string)
 */
function winnerName() public view
    returns (string memory winnerName_)
{
    winnerName_ = proposals[winningProposal()].name;
}
}
```

Listing 1. Ballot.sol

Output

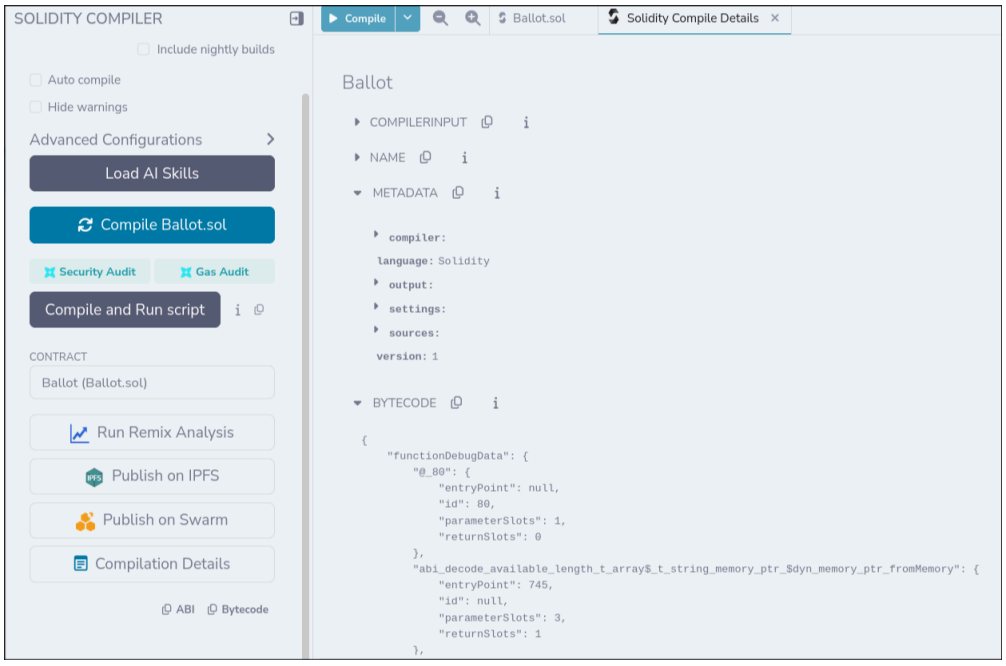


Fig. 1. Compiled Ballot contract details

Experiment No. 6

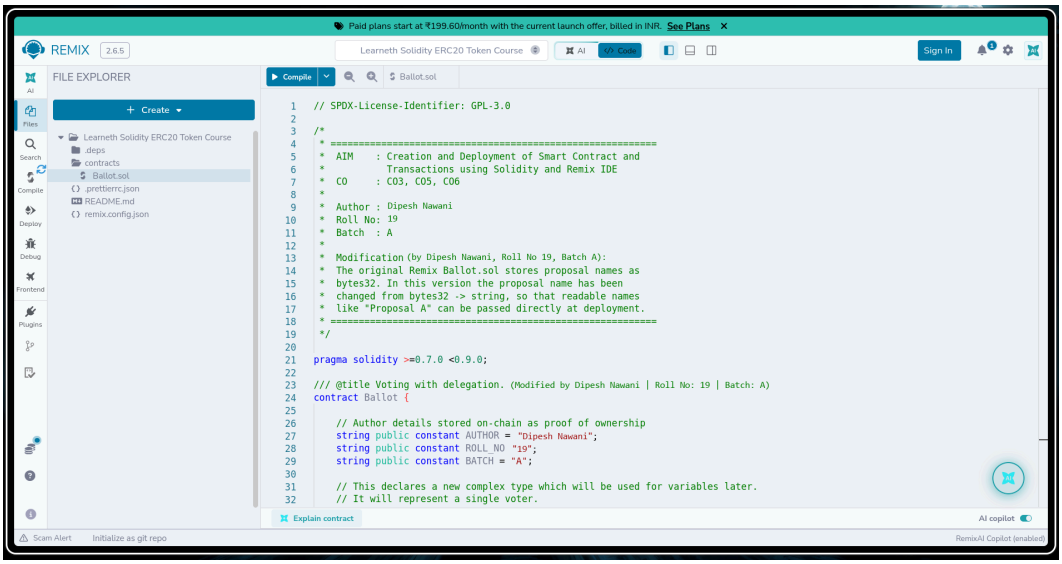


Fig. 2. Ballot source code in Remix IDE

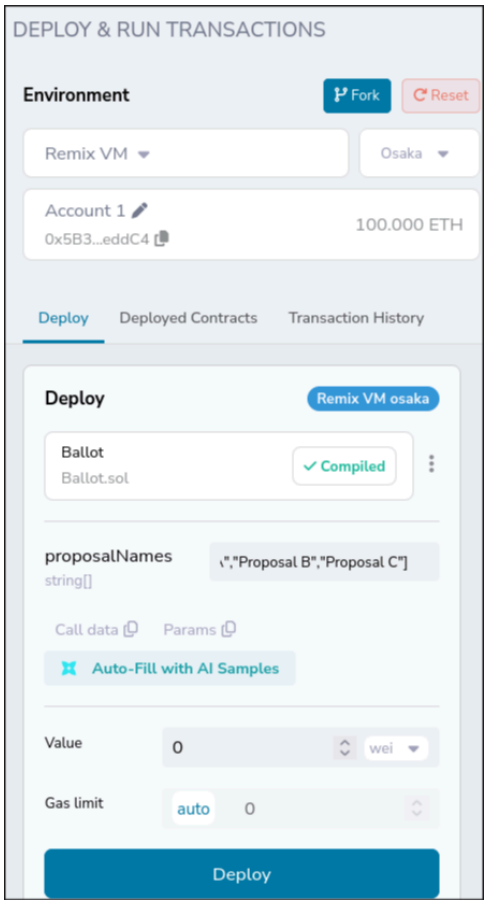


Fig. 3. Deployment form with proposal names

Experiment No. 6

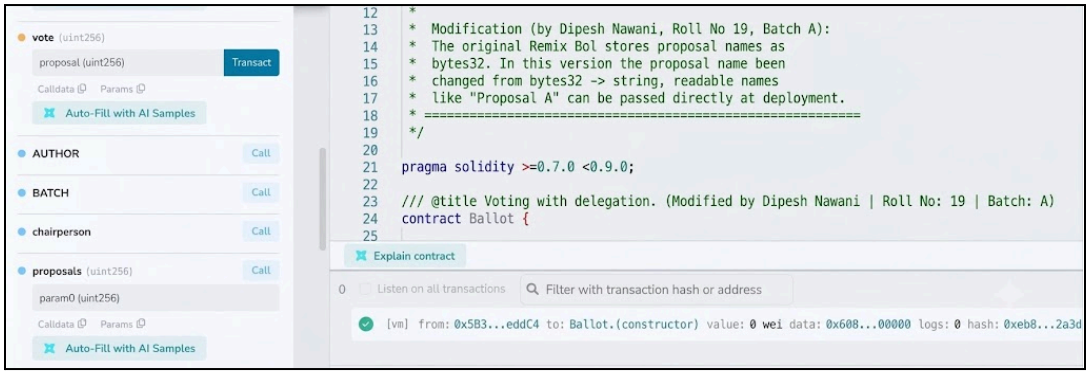


Fig. 4. Successful Ballot deployment transaction

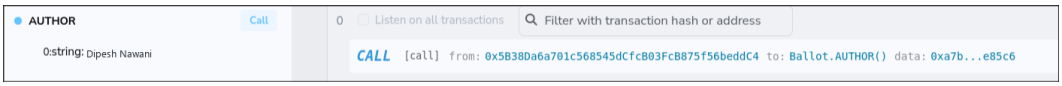


Fig. 5. Author value returned by contract call

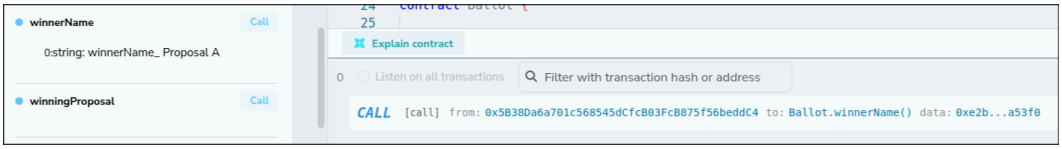


Fig. 6. Winner name returned by contract call

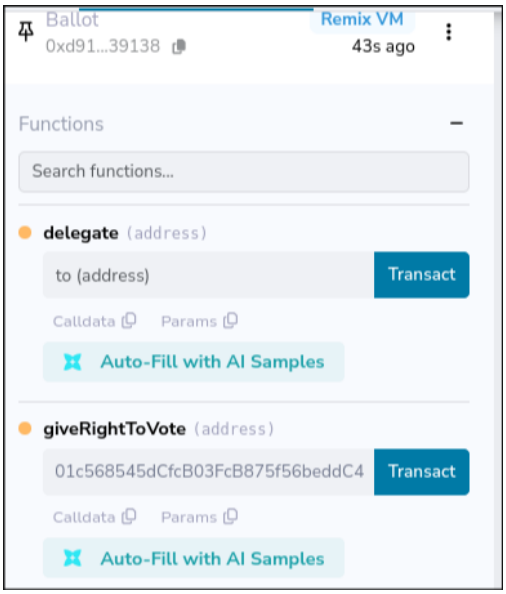


Fig. 7. Delegation and voting functions available

Experiment No. 6

Conclusion

The Ballot smart contract was successfully created, compiled, deployed, and validated in Remix IDE using the modified proposal-name representation. The deployment utilized human-readable candidate strings ("Proposal A", "Proposal B", "Proposal C") directly within the constructor, generating an authentic on-chain deployment transaction in the Remix VM environment.

Contract calls accurately returned the embedded author ownership details ("Dipesh Nawani", Roll No. 19, Batch A) as well as the winning proposal output from decentralized consensus. The deployed user interface properly exposed and executed voting and delegation functions with complete state synchronization. The experiment effectively demonstrated the complete lifecycle of developing, compiling, deploying, and transacting with Solidity smart contracts on the Ethereum Virtual Machine.